



August 2026

Prepared for
Paxos Labs

Audited by
devtooligan
ret2basic

Paxos Labs PR 324

Smart Contract Security Assessment

Contents

1	Review Summary	2
1.1	Protocol Overview	2
1.2	Audit Scope	2
1.3	Risk Assessment Framework	2
1.3.1	Severity Classification	3
1.4	Key Findings	3
1.5	Overall Assessment	3
2	Audit Overview	4
2.1	Project Information	4
2.2	Audit Timeline	4
2.3	Audit Resources	4
2.4	Critical Findings	6
2.5	High Findings	6
2.5.1	A flash loan’s nested batch never enters the dangling-approval sweep . . .	6
2.6	Medium Findings	11
2.7	Low Findings	11
2.7.1	<code>_forceProcess</code> omits every guard <code>processOrders</code> has, so the admin override fails in the situations it exists for	11
2.7.2	The dangling-approval sweep does not cover <code>increaseApproval</code> , the allowance-raising function <code>PAXG</code> and <code>USDG</code> implement	15
2.7.3	Deployment salts encode a zero deployer address, allowing anyone to front run the deployment	18
2.8	Gas Savings Findings	22
2.9	Informational Findings	22
2.9.1	<code>execute</code> ’s snapshot comment claims an alignment guarantee the code does not provide	22
2.9.2	The subsidy is credited at the amount sent rather than the amount received, and the contract carries no unsupported-token disclaimer	24
2.9.3	The <code>_unitRate</code> divide-branch safety argument does not hold for a mixed basket	26
2.9.4	<code>PaxgXauRateProvider</code> bounds each feed’s age but never the skew between them	28
2.9.5	<code>UManager</code> ’s <code>_boringVault</code> constructor argument is redundant	33
2.9.6	Design documentation reconciliation	34
2.9.7	<code>Permit2</code> allowances can persist	36

1 Review Summary

1.1 Protocol Overview

PR #324 introduces the EquivalentExchangeUManager for the PAXGy Funds Automation Vault. It executes Merkle-verified vault action batches, checks per-token balance deltas, preserves the aggregate value of a configured token basket, and pulls subsidy when a batch creates a shortfall. The pull request also adds PAXG/XAU pricing, dynamic deposit and withdrawal fee modules, deployment scripts, DEX UManagers, and their supporting tests.

1.2 Audit Scope

This audit covers all changes in PR #324 of the pxl-boring-vault repository at commit 0af42540fb25a1e5ea85b7142adb4f3a3b7954d5, comprising 22 changed files.

```
.
├── .husky
│   └── post-commit
├── script
│   └── deploy
│       ├── DeployBoringVaultAndManager.s.sol
│       └── DeployPaxgXauRateProvider.s.sol
├── src
│   ├── helper
│   │   ├── PaxgyDynamicDepositFeeModule.sol
│   │   └── PaxgyDynamicWithdrawalFeeModule.sol
│   ├── micro-managers
│   │   ├── DexAggregatorUManager.sol
│   │   ├── DexSwapperUManager.sol
│   │   ├── EquivalentExchangeUManager.sol
│   │   └── UManager.sol
│   └── oracles
│       └── PaxgXauRateProvider.sol
└── test
    ├── PaxgXauRateProvider.t.sol
    ├── PaxgyDynamicDepositFeeModule.t.sol
    ├── PaxgyDynamicWithdrawalFeeModule.t.sol
    ├── micro-managers
    │   ├── DexAggregatorUManager.t.sol
    │   ├── DexSwapperUManager.t.sol
    │   ├── EEUManagerDeployWiring.t.sol
    │   ├── EquivalentExchangeUManager.t.sol
    │   ├── EquivalentExchangeUManagerIntegration.t.sol
    │   ├── EquivalentExchangeUManagerInternal.t.sol
    │   └── mocks
    │       └── MockRateProvider.sol
    └── mocks
        ├── MockPriceFeed.sol
        └── MockToken.sol
```

1.3 Risk Assessment Framework

1.3.1 Severity Classification

Severity	Description	Potential Impact
Critical	Immediate threat to user funds or protocol integrity	Direct loss of funds, protocol compromise
High	Significant security risk requiring urgent attention	Potential fund loss, major functionality disruption
Medium	Important issue that should be addressed	Limited fund risk, functionality concerns
Low	Minor issue with minimal impact	Best practice violations, minor inefficiencies
Undetermined	Findings whose impact could not be fully assessed within the time constraints of the engagement. These issues may range from low to critical severity, and although their exact consequences remain uncertain, they present a sufficient potential risk to warrant attention and remediation.	Varies based on actual severity
Gas	Findings that can improve the gas efficiency of the contracts.	Increased transaction costs
Informational	Code quality and best practice recommendations	Reduced maintainability and readability

Table 1: severity classification

1.4 Key Findings

Breakdown of Finding Impacts

Impact Level	Count
■ Critical	0
■ High	1
■ Medium	0
■ Low	3
■ Informational	7

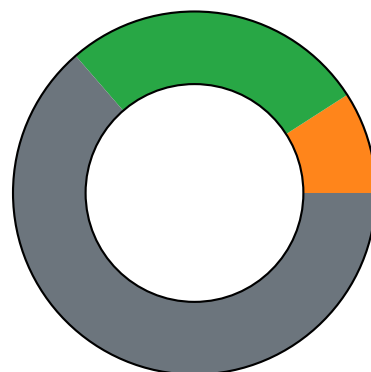


Figure 1: Distribution of security findings by impact level

1.5 Overall Assessment

The review reports 11 findings: 1 High, 3 Low, and 7 Informational. The principal risks were an uninspected nested flash-loan batch that could leave standing token approvals, deployer-unprotected CreateX salts, and oracle freshness and timing assumptions. Seven

findings were remediated in follow-up PR #342, two were acknowledged, one was partially remediated, and one documentation finding was reported as fixed but could not be independently verified. PR #342 remains open, so its fixes are not yet present in PR #324 or master.

2 Audit Overview

2.1 Project Information

Protocol Name: Paxos Labs

Repository: <https://github.com/paxoslabs/pxl-boring-vault>

Commit Hash: 0af42540fb25a1e5ea85b7142adb4f3a3b7954d5

Final Commit Hash: ae5c652339ba0c144fad61b752f8581188dd1215

Commit URL: <https://github.com/paxoslabs/pxl-boring-vault/pull/324>

2.2 Audit Timeline

The audit was conducted from August 5 to 15, 2026.

2.3 Audit Resources

- PR #324 source code, deployment scripts, and tests
- Equivalent Exchange and PAXGy fee-module design documentation
- Developer responses recorded in the yAudit issue repository
- Follow-up remediation PR #342

Category	Mark	Description
Access Control	Average	Owner, authority, strategist, and Merkle-root boundaries are generally explicit, and basket reconfiguration remains owner-only. However, the manager flash-loan path created a nested execution layer outside the UManager’s approval inspection, and the approval selector coverage did not match two deployed Paxos tokens.
Mathematics	Average	Basket normalization, value aggregation, and subsidy rounding are structured to fail closed in common configurations. The review nevertheless identified an unsafe lossy scaling branch for mixed baskets and an unbounded skew between the two feeds the PAXG/XAU rate is composed from.
Complexity	Average	The design combines parallel calldata arrays, bitmap-indexed oracle configuration, Merkle authorization, nested manager execution, subsidy accounting, and dynamic withdrawal processing. These interacting layers increase review and maintenance burden and contributed directly to approval-tracking and documentation gaps.
Libraries	Good	The implementation uses established Solmate, OpenZeppelin, CreateX, and protocol interface dependencies. Library selection was generally appropriate, although the deployment script’s initialization order unintentionally disabled the deployer-prefixed protection expected from CreateX salts.
Decentralization	Low	Operation depends heavily on a Paxos multisig, authorized strategists, backend timing decisions, a configured subsidy payer, and controlled Merkle roots. These are documented product assumptions, but they create meaningful centralization and liveness dependencies.
Documentation	Average	The code contains extensive NatSpec and security-assumption commentary, but the review found discrepancies in the exported design documentation, an incorrect snapshot-alignment guarantee, an overbroad WithdrawQueue claim, and a missing unsupported-token disclaimer. Several code comments were corrected in PR #342, while the revised design PDF was not available for verification.
Testing and verification	Average	The pull request includes broad unit and integration coverage for valuation, fees, deployment wiring, and approval cleanup. Important production-specific paths were absent: PAXG and USDG expose increaseApproval rather than increaseAllowance, nested flash-loan batches and Permit2 persistence were not exercised, and follow-up PR #342 omitted a mock required by its tests.

Table 2: Code Evaluation Matrix

2.4 Critical Findings

None.

2.5 High Findings

2.5.1 A flash loan's nested batch never enters the dangling-approval sweep

`_enforceNoDanglingApprovals` iterates the outer batch's `calls.targets` and drops anything that is not a basket token. `receiveFlashLoan` decodes a complete, independent batch out of `userData` and executes it, so a batch whose only outer action is `manager.flashLoan(...)` presents `calls.targets == [manager]` to the sweep and is skipped entirely, while an `ERC20.approve(spender, type(uint256).max)` sitting inside `userData` is never examined. Compounding this, `userData` is not committed to the outer Merkle leaf at all — the decoder discards it — so a single `flashLoan` leaf authorises an unconstrained second batch whose contents the sweep cannot see.

Technical Details

`_enforceNoDanglingApprovals` iterates the outer batch's `calls.targets` and drops anything that is not a basket token:

```
1 address target = calls.targets[i];
2 if (!basketTokens.contains(target)) continue;
```

`receiveFlashLoan` decodes a complete, independent batch out of `userData` and executes it:

```
1 (
2     bytes32[][] memory manageProofs,
3     address[] memory decodersAndSanitizers,
4     address[] memory targets,
5     bytes[] memory data,
6     uint256[] memory values
7 ) = abi.decode(userData, (bytes32[][], address[], address[], bytes[], uint256[]));

9 ManagerWithMerkleVerification(address(this))
10     .manageVaultWithMerkleVerification(manageProofs, decodersAndSanitizers, targets,
    data, values);
```

If the outer batch's only action is `manager.flashLoan(...)`, then `calls.targets == [manager]`. The manager is not a basket token, so the sweep skips the whole batch at its first filter, and an `ERC20.approve(spender, type(uint256).max)` inside `userData` is never examined. The sweep only ever sees the one layer of calldata the UManager forwarded; the inner batch is a different layer.

`userData` is not committed to the leaf

This is what makes the outer leaf load-bearing rather than incidental. `BalancerV2DecoderAndSanitizer.flashLoan` drops its fourth argument:

```

1 function flashLoan(address recipient, address[] calldata tokens, uint256[] calldata,
  bytes calldata)
2     external pure virtual returns (bytes memory addressesFound)
3 {
4     addressesFound = abi.encodePacked(recipient);
5     for (uint256 i; i < tokens.length; ++i) {
6         addressesFound = abi.encodePacked(addressesFound, tokens[i]);
7     }
8 }

```

The leaf therefore commits to `recipient` and the token list and says nothing whatsoever about what the callback does. One `flashLoan` leaf is a general-purpose door to "execute an arbitrary second batch under `manageRoot[manager]`", and everything behind that door is invisible to EE's sweep. This is a property of the design, not of any particular leaf list.

Full path

Vault holds basket tokens. Production root contains two ordinary leaves: L1 = `flashLoan` on the manager, L2 = `approve(VENUE, *)` on a basket token.

strategist

```

└ EE.execute(calls, payer, USDC, allowableTokenDelta = [0,0,0])
  | calls.targets = [manager]
  | calls.targetData = [flashLoan(manager, [USDC], [0], userData)]
  | userData = abi.encode(inner batch: [USDC.approve(VENUE, 2^256-1)])
  |
  |─ snapshot balances
  |─ manager.manageVaultWithMerkleVerification(outer) msg.sender = EE -> manageVault
  |   └ vault.manage(manager, flashLoan(...), 0)
  |       └ manager.flashLoan(...) gate is only `msg.sender == address(vault)` ✓
  |           └ balancerVault.flashLoan(manager, [USDC], [0], userData)
  |               └ manager.receiveFlashLoan(...) msg.sender = balancerVault ✓
  |                   └ manager.manageVaultWithMerkleVerification(inner)
  |                       | msg.sender = manager -> manageVault
  |                       └ vault.manage(USDC, approve(VENUE, 2^256-1), 0)
  |                           ⇒ allowance[vault][VENUE] = 2^256-1 <- approval lands
  |
  |─ _checkAndValueBasket: no balance changed -> deltas 0, totalAfter == totalBefore ✓
  |─ shortfall = 0 -> subsidy pulled = 0
  |─ _enforceNoDanglingApprovals(calls)
  |   └ calls.targets = [manager] -> not a basket token -> continue -> no revert

```

Executed reports `totalBefore == totalAfter` and `subsidyAmount == 0`: a textbook no-op.

In a second, unrelated transaction, `VENUE` calls

`transferFrom(vault, attacker, balanceOf(vault))` — an ordinary ERC20 call that passes through neither the UManager nor the Merkle manager, so no snapshot, no delta bound, no value invariant and no subsidy applies. The demonstration borrows zero; a real batch would borrow a real amount, repay it normally, and carry the extra `approve` alongside. The bypass is identical either way, because the sweep looks at the outer `calls.targets` and is indifferent to the loan.

Reproduced

Executed against a mainnet fork at block 25,654,305, using the **deployed** contracts (vault `0x626b...c022`, manager `0x1112...0B53`, EE `0x702c...1624`, authority `0xC051...8969`) and the **production** `GenericDecoderAndSanitizer` — no purpose-built decoder:

[PASS] `test_NestedFlashLoanBatchEscapesDanglingApprovalSweep()` (gas: 562253)

The test enables flash loans with the two settings described below, installs a single-leaf `flashLoan` root on EE and a single-leaf `approve` root on the manager, and submits an outer batch whose only target is the manager. `EE.execute` succeeds — the value invariant sees no balance change and the sweep skips the batch — and then:

```
1 assertEq(PAXG.allowance(PFAV, SPENDER), AMOUNT);           // approval survived the sweep
2 vm.prank(SPENDER);
3 assertTrue(PAXG.transferFrom(PFAV, ATTACKER, AMOUNT));    // drained in a later,
   unguarded tx
```

The loan amount is zero in the test, which shows the bypass does not depend on the loan at all — the sweep reads the outer `calls.targets` and is indifferent to what is borrowed.

Why two roots is not an obstacle

The inner call is a self-call, so it verifies against `manageRoot[address(manager)]`, a different root from `manageRoot[EE]`. The repository’s own flash-loan test installs *the same root* on both principals, with a comment saying that is required:

[test/ManagerWithMerkleVerification.t.sol:172-174](#). That same test’s inner batch consists of exactly two `ERC20.approve` calls. `approve`’s amount is also not committed to its leaf, so an authorised `(token, spender)` pair can be approved for `type(uint256).max`.

Preconditions, and what the severity turns on

Verified on the live deployment at the time of writing (manager `0x1112D39Cbe51534499BED6A169140e631AD70B53`, authority `0xC051ab4Cf4Ae8314CEad1126208CfFE96Bd38969`):

```
manager.balancerVault()           = 0xBA12222222228d8Ba445958a
canCall(EE, manager, manageVaultWithMerkleVerification) = true
canCall(manager, manager, manageVaultWithMerkleVerification) = false <-- self-call not
manageRoot[manager]               = 0x00...00
manageRoot[EE]                   = 0x00...00
```

So the inner self-call would revert at `requiresAuth` today. This path needs strictly more than the baseline that gates every finding in this review, and that increment is a real likelihood discount which we do not argue away.

But the increment is two settings, not four. Granting the manager a role over its own `manageVaultWithMerkleVerification` and setting `manageRoot[manager]` are jointly indivisible — together they *are* “flash loans are enabled”, and neither alone does anything. The other two items commonly listed as separate gates are not independent:

- **The inner `approve` leaf comes with the root.** The reference configuration installs *the same tree* on both principals, with a comment stating why: `test/ManagerWithMerkleVerification.t.sol:170-174` — “Since the manager calls to itself to fulfill the flashloan, we need to set its root.” Under that pattern `manageRoot[manager] == manageRoot[EE]`, so every `approve` leaf in the outer root is automatically in the inner one. Nobody has to enable an “approval route” separately.
- **Approves in the inner batch are the documented usage.** That same test’s inner batch is literally two `ERC20.approve` calls (lines 184-185). `approve` is also the most common leaf type in the suite, and `type(uint256).max` approves are routine in it. Expecting a production root with no `approve` leaves is unrealistic — the existence of EE’s dangling-approval sweep is itself an admission that `approve` leaves are expected.

On provenance, stated precisely, because it bears on how much intent to read into the wiring. The flash-loan mechanism itself is inherited from the upstream BoringVault fork, not written by Paxos: `receiveFlashLoan` arrives in `e48e043`, and every subsequent commit touching that file is formatting or natspec. The reference flash-loan test is upstream too. So “the repository’s own test does exactly this” describes the framework’s canonical usage, not a Paxos decision.

What *is* Paxos’s own decision is the EE deploy script they wrote: it hardcodes `BALANCER_VAULT = 0xBA12...` and passes it into the constructor rather than `address(0)`, which would have disabled the feature permanently (`receiveFlashLoan` requires `msg.sender == balancerVault`). The script’s own comment lists `BALANCER_VAULT` among the mainnet-specific addresses it deliberately verified, alongside USDC, USDC, PAXG and the price feed — so it was considered, not inherited by inattention. The live manager confirms the result.

The decisive point is narrower than “the feature is provisioned”, and does not depend on reading intent at all: **this path requires no code that does not already exist and is not already deployed.** `BalancerV2DecoderAndSanitizer.flashLoan` is in `GenericDecoderAndSanitizer`’s inheritance list, `BaseDecoderAndSanitizer.approve` is there, and `receiveFlashLoan` is live in the deployed manager. Only settings are missing.

This rating is falsifiable and we would like it tested. It rests on one question the team can answer definitively: *will the production EE root contain a `manager.flashLoan` leaf, and will `manageRoot[manager]` be set?* If yes, High stands. If the answer is that EE will never open a second layer, Medium is the right rating — provided that constraint is written into the leaf-review checklist rather than left as an implicit assumption, since nothing in the code enforces it.

Impact

High. An approval created inside a flash-loan callback survives the batch and is spendable later, in a transaction that passes through neither the UManager nor the Merkle manager, so it receives no balance snapshot, no delta bound, no value invariant and no subsidy top-up. Because `approve`’s amount is not committed to the leaf, the surviving allowance can be unbounded and the ceiling is the vault’s full balance of that token.

Unlike #2, no Permit2 integration and no pre-existing ERC20 allowance is required: the approval is an ordinary ERC20 allowance on a basket token, created inside a batch invisible to the sweep. Unlike #7, no new decoder code is required either —

`BalancerV2DecoderAndSanitizer.flashLoan` and `BaseDecoderAndSanitizer.approve` are both already in the production decoder.

Reachable by a compromised strategist, and usable by a venue that is upgraded or turns malicious after being Merkle-approved — the case the sweep exists to neutralise. The `approve` leaf it needs is near-certain to be in any production root for a swap-executing manager; the existence of the dangling-approval sweep is itself evidence that approve leaves are expected.

This is one of three instances of a single root cause: the sweep is a calldata pattern match rather than an assertion about allowance state. #2 escapes by changing the *target* (Permit2 is not a basket token), #7 by changing the *selector*, this one by changing the *call layer*.

Recommendation

Replace the calldata scan with an allowance-state assertion, which closes this path and #2 together. Snapshot `allowance(boringVault, spender)` for each (basket token × watched spender) pair before `_manageVaultWithMerkleVerification`, re-read after, and require

```
allowanceAfter <= allowanceBefore
```

The check is indifferent to which layer created the approval, so a nested flash-loan batch is covered without the sweep having to understand `receiveFlashLoan` at all. "Must not increase" rather than "must be zero" preserves legitimate standing approvals that predate the batch. This needs an owner-configured `watchedSpenders` set with the same multisig-only access as `basketTokens`, and its maintenance must be bound into Merkle root rotation: any spender entering the root must simultaneously enter `watchedSpenders`.

If the sweep is kept as-is, the most practical targeted mitigation is to **stop sharing one root across both principals**. The reference pattern sets `manageRoot[manager] == manageRoot[EE]` for convenience; setting a separate, deliberately narrow inner root containing only what a flash-loan callback actually needs — and no `approve` leaves — closes this path directly. It is feasible because repayment does not need an approval: `receiveFlashLoan` returns the borrowed amount via `vault.manage(tokens, transferData, ...)`, a direct `ERC20.transfer`. The cost is that whatever the borrowed funds are used for must fit inside that narrow root, which is a real constraint on flash-loan strategies and should be decided deliberately rather than by copying the test.

Two further options, either as alternatives or in depth: keep `manager.flashLoan` out of `manageRoot[EE]` entirely so EE batches can never open a second layer; or commit `keccak256(userData)` to the outer leaf, which closes the "unconstrained second batch" property though not the sweep's blindness to nested layers in general. Making the sweep decode `manager.flashLoan` and recurse into the embedded batch also works but re-solves the problem in the same brittle style that produced #2 and #7.

Whichever is chosen, the constraint must be written into the leaf-review checklist rather than left implicit — nothing in the code enforces any of them.

Either way, audit the production leaf list for `flashLoan` leaves before launch, and add a test asserting that an approval created inside a flash-loan callback is rejected. The current dangling-approval tests only exercise single-layer batches.

Developer Response

Fixed. The built-in flash loan function is now blocked in the context of the Equivalent Exchange contract.

2.6 Medium Findings

None.

2.7 Low Findings

2.7.1 `_forceProcess` omits every guard `processOrders` has, so the admin override fails in the situations it exists for

`forceProcess` / `forceProcessOrders` are the operator's escape hatch for orders that the normal queue cannot clear. Their shared implementation `_forceProcess` (`src/base/Roles/WithdrawQueue.sol:632-649`) is a stripped-down copy of the `processOrders` body: it performs the same fee call and the same withdraw, but without any of the four protections the loop in `processOrders` carries.

Guard in <code>processOrders</code>	Line
<code>if (tellerWithMultiAssetSupport.isPaused()) revert TellerIsPaused();</code>	:501-
<code>if (feeAmount >= order.amountOffer) { ... refund ... }</code>	:512-
VaultInsufficientBalance precheck	:531-
<code>try tellerWithMultiAssetSupport.bulkWithdraw(...) { ... } catch { _refundOrder(...) }</code>	:541-

Two consequences follow, and they are the same defect seen from two directions:

1. **The override does not survive a stale oracle.** `_forceProcess` calls `feeModule.calculateOfferFees` at :641 with no `try/catch`. Both Paxgy fee modules revert when `PaxgXauRateProvider` reports a stale feed, so `forceProcess` reverts for exactly the same reason `processOrders` does. The team's stated fallback for a stale-price stall — *"worst case, we have admin overrides in the withdrawal queue"* (Nick 41:29) — is not available in that state.
2. **The override does not survive a frozen receiver.** The `try/catch` that `_forceProcess` lacks is, per `_refundOrder`'s own NatSpec (:664-669), there *"so that a frozen or blacklisted refundReceiver does not revert the entire processOrders loop."* PAXG and USDG both carry blocklists. Where `processOrders` catches and refunds, `_forceProcess` reverts — failing precisely in the scenario it was written to rescue.

The in-scope hook is `PaxgyDynamicWithdrawalFeeModule`, whose NatSpec states the refund behaviour as unconditional when it only holds on one of the two paths.

Scope note: `WithdrawQueue.sol` is outside the four files in `SCOPE.md`. The incorrect assertion in `src/helper/PaxgyDynamicWithdrawalFeeModule.sol:148-163` is in scope.

Technical Details

The two implementations, side by side

```

1 // processOrders - src/base/Roles/WithdrawQueue.sol:501-559 (abridged)
2 if (tellerWithMultiAssetSupport.isPaused()) revert TellerIsPaused();
3 address receiver = ownerOf(orderIndex);
4 _burn(orderIndex);
5 uint256 feeAmount = feeModule.calculateOfferFees(order.amountOffer, offerAsset, order.
  wantAsset, receiver);
6 if (feeAmount >= order.amountOffer) {                                // (a) cap enforcement
7     orderAtQueueIndex[orderIndex].orderType = OrderType.REFUND;
8     _refundOrder(order, orderIndex);
9     emit FeesExceededOrderAmount(orderIndex, order.amountOffer, feeAmount);
10    continue;
11 }
12 ...
13 if (vaultBalanceOfWantAsset < expectedAssetsOut) revert VaultInsufficientBalance(...);
14 // (b)
15 try tellerWithMultiAssetSupport.bulkWithdraw(...) returns (uint256 assetsOut) {
16 // (c)
17     ...
18 } catch {
19     orderAtQueueIndex[orderIndex].didOrderFailTransfer = true;
20     _refundOrder(order, orderIndex);
21 }

```

```

1 // _forceProcess - src/base/Roles/WithdrawQueue.sol:632-649 (complete)
2 function _forceProcess(uint256 orderIndex) internal {
3     Order memory order = _getOrderEnsureDefault(orderIndex);
4     orderAtQueueIndex[orderIndex].orderType = OrderType.PRE_FILLED;
5     address receiver = ownerOf(orderIndex);
6     _burn(orderIndex);
7
8     uint256 feeAmount = feeModule.calculateOfferFees(order.amountOffer, offerAsset,
  order.wantAsset, receiver);
9     uint256 assetsOut = tellerWithMultiAssetSupport.bulkWithdraw(
10         ERC20(address(order.wantAsset)), order.amountOffer - feeAmount, 0, receiver
11     );
12
13     offerAsset.safeTransfer(feeRecipient, feeAmount);
14     emit OrderProcessed(orderIndex, orderAtQueueIndex[orderIndex], receiver, true,
  feeAmount, assetsOut);
15 }

```

Missing guard (a) means `order.amountOffer - feeAmount` underflows and reverts (checked arithmetic, so no fund loss) where `processOrders` would have refunded the order. Missing guard (c) means a `bulkWithdraw` that fails on a blocklisted `receiver` reverts the whole call instead of routing the order to `_refundOrder` and, if the refund transfer also fails, to `recoveryAddress`.

No path completes a queued withdrawal during a stale-oracle window

Every entrypoint that would settle an already-queued order calls the same fee module:

Entrypoint	fee call	Behaviour on stale feed
<code>processOrders</code>	<code>:509</code>	reverts (call is outside the <code>try</code>)
<code>forceProcess</code> → <code>_forceProcess</code>	<code>:641</code>	reverts
<code>forceProcessOrders</code> → <code>_forceProcess</code>	<code>:641</code>	reverts
<code>submitOrderAndProcess*</code>	via <code>processOrders</code>	reverts

Repointing the queue at a different fee module is not an escape either: `setFeeModule` requires `totalSupply() == 0` (`:214-220`), i.e. an empty queue, and the queue can only be emptied by refunding.

The one administrative action that still works is `refundOrders`, which returns shares to users. That unblocks the queue but does not complete anybody's withdrawal — users end up back where they started, holding shares rather than PAXG. If "admin override" was understood to mean "force the withdrawal through", that capability does not exist in this state.

(A stronger variant was considered and disproved: `setFeeModule`'s `QueueMustBeEmpty` does *not* create a permanent deadlock. The `REFUND` branch of `processOrders` continues at `:489-497`, before the `calculateOfferFees` call, so refunds never touch the fee module and the queue can always be drained.)

The in-scope module asserts the guard unconditionally

`PaxgyDynamicWithdrawalFeeModule` documents its cap by pointing at a consumer-side behaviour that only two of its four consumers implement:

```

1 // src/helper/PaxgyDynamicWithdrawalFeeModule.sol:148-151
2 // ... the fee scales directly with the reported PAXG:XAU premium, so a genuine market
  depeg charges the
3 // withdrawer the full premium (bounded only by the order amount, above which
  WithdrawQueue refunds the
4 // order). ...

6 // :160-163
7 // Capping preserves the fee <= amount invariant the WithdrawQueue relies on.
```

Consumers of `calculateOfferFees` and whether they enforce `fee < amount`:

Consumer	<code>fee >= amount</code> guard	failure fallback
<code>DistributorCodeDepositor._deposit:274</code>		atomic; user can retry
<code>WithdrawQueue.processOrders:512</code>	marks <code>REFUND</code> and refunds	<code>try/catch</code>
<code>WithdrawQueue._forceProcess:641</code>		
<code>OneToOneQueue.submitOrder:442</code>		—

Reachability, narrowed honestly

Under the paxgy configuration (`minimumOrderSize = 1e15`), triggering the missing cap through the *fee* path requires an extreme PAXG:XAU distortion, and the dust path is blocked by the minimum order size. The practically reachable sub-case is the **blocklisted receiver**, which needs no price anomaly at all: PAXG and USDG are blocklist-bearing tokens, and `_refundOrder`'s own comment names frozen receivers as the scenario the `try/catch` exists for.

Impact

Operational, not a fund loss:

- The documented administrative fallback is unavailable in the two states that most plausibly require it — a stale price feed, and a receiver who has been frozen since the order was placed. In both, `forceProcess` reverts, and the only remaining action is a refund.
- `_forceProcess` and `processOrders` disagree on how the same order is handled, which is a maintenance hazard independent of any specific trigger.
- The fee module's NatSpec records a safety property it does not universally have, so a future reader may reason from a guarantee that holds on only half the paths.

No user funds can be taken. The considered variant "a zero-share withdrawal lets the whole order be seized as fees" was checked and is **false**:

`TellerWithMultiAssetSupport.bulkWithdraw` reverts with `ZeroShares()` when `shareAmount == 0`, so execution never reaches the transfer to `feeRecipient`.

Recommendation

1. Bring `_forceProcess` to parity with `processOrders`, ideally by extracting the shared body into a single internal function parameterized by the "skip position check" behaviour, so the two can no longer drift. At minimum add the `feeAmount >= order.amountOffer` refund branch and the `try/catch` around `bulkWithdraw`.
2. Move `calculateOfferFees` inside a `try/catch` (or make it per-order skippable) so one stale read cannot revert an entire batch, and so an administrator retains a way to settle orders that do not depend on the dynamic fee.
3. Correct `PaxgyDynamicWithdrawalFeeModule`'s NatSpec so the refund claim is scoped to `processOrders` rather than to "WithdrawQueue" generally.
4. Confirm with operations what "admin override" is expected to accomplish during a stale-price window, and document the actual available actions — currently that is `refundOrders` only.

Developer Response

Fixed. The comment which made these specific claims has been updated.

2.7.2 The dangling-approval sweep does not cover `increaseApproval`, the allowance-raising function PAXG and USDG implement

`_enforceNoDanglingApprovals` recognises exactly two selectors, `approve(address,uint256)` and `increaseAllowance(address,uint256)`. Probed against mainnet, `increaseAllowance` **does not exist on two of the three basket tokens**: PAXG and USDG are Paxos-issued tokens that expose the older `increaseApproval(address,uint256)` instead. The guard's second arm is therefore dead code on this deployment, and the allowance-raising function those two tokens actually implement is not in the selector set. The guard covers less than it appears to, today, in shipped code.

This is a latent gap rather than an armed one: no decoder in the repository exposes `increaseApproval`, so no leaf can currently authorise it. That precondition is stated up front because it is what caps the rating at Low despite a high impact ceiling, and it is what separates this from #8 — see the severity discussion below.

Technical Details

`_enforceNoDanglingApprovals` recognises exactly two selectors:

```
1 bytes4 selector = bytes4(targetData[0:4]);
2 if (selector != ERC20.approve.selector && selector != INCREASE_ALLOWANCE_SELECTOR)
   continue;
```

where `INCREASE_ALLOWANCE_SELECTOR` is `0x39509351`, i.e.

`increaseAllowance(address,uint256)`. Its presence is evidence of intent: the author deliberately set out to cover allowance-raising entrypoints beyond `approve`. The defect is not an omission of the idea, it is that the wrong function name was chosen for this basket.

Probed against mainnet:

Token		<code>increaseAllowance(address,uint256)</code> <code>0x39509351</code>
PAXG <code>0x45804880De22913dAFE09f4980848ECE6EcbAf78</code>		reverts, empty returndata
USDG <code>0xe343167631d89B6Ffc58B88d6b7fB0228795491D</code>		reverts <code>0x800ab12c</code>
USDC <code>0xA0b86991c6218b36c1D19D4a2e9Eb0cE3606eB48</code>		returns true

Distinguishing "function missing" from "function reverted": PAXG returns empty returndata for a selector that certainly does not exist (`qqqAlsoMissing()`), and the same for `increaseAllowance`. `increaseApproval` with `addedValue = 0` reverts `0x7c946ed7 = ZeroValue()`, an internal argument check, so dispatch matched; with a non-zero amount it returns `true`. USDG behaves identically except that its unknown-selector error is `0x800ab12c`.

```
1 W=0x28C6c06298d514Db089934071355E5743bf21d60 # any funded EOA, used only as --from
3 cast call 0x45804880De22913dAFE09f4980848ECE6EcbAf78 "increaseApproval(address,uint256)(
  bool)" \
4 0x0000000000000000000000000000000000000000000000000000000000000000dEaD 100000000000000000000000000000000000000000000000000000000000000000 --from $W --rpc-url
  $RPC
5 # true
```

```
7 cast call 0x45804880De22913dAFE09f4980848ECE6EcbAf78 "increaseAllowance(address,uint256)
  " \
8   0x0000000000000000000000000000000000000000000000000000000000000000daD 1 --rpc-url $RPC
9 # reverts, empty returndata
```

A batch containing `PAXG.increaseApproval(spender, type(uint256).max):`

1. changes no basket balance, so `_checkTokenDelta` and the value invariant both pass;
2. is skipped at the selector filter;
3. leaves a live, unbounded, ordinary ERC20 allowance on a basket token, spendable in a later transaction that passes through neither the UManager nor the Merkle manager.

Reproduced against a mainnet fork at block 25,654,305 using the deployed vault, manager and EE: with a leaf authorising `PAXG.increaseApproval`, `EE.execute` succeeds, `PAXG.allowance(vault, spender)` is non-zero afterwards, and `transferFrom` drains the balance in a later transaction that touches neither contract.

The project's tests for this sweep — `test_Execute_RevertWhen_DanglingApprovalLeft` and `test_Execute_RevertWhen_DanglingApprovalViaIncreaseAllowance` — exercise a `MockERC20` that implements `increaseAllowance`. The mock's allowance surface does not match the tokens actually in the basket, which is why the gap is invisible from the suite. The second test passes while asserting coverage that does not exist on two thirds of the real basket.

This also removes the premise that keeps #2 inert. #2 rates persisting Permit2 grants Informational because "the ERC20 leg targets a basket token with the `approve` selector, so the sweep does check it and reverts the batch if a non-zero allowance to Permit2 remains at the end." That holds only while the ERC20 leg is expressed as `approve`.

`PAXG.increaseApproval(Permit2, X)` establishes the same first-layer allowance and is not swept, at which point every Permit2 grant accumulated earlier becomes live for its full amount until its stored expiration.

On the severity

The impact ceiling is the same as #8's — an unbounded standing allowance on a basket token, with the vault's full balance of that token as the loss ceiling. What holds the rating at Low is that reaching it requires a decoder exposing `increaseApproval`, and no such decoder exists: `BaseDecoderAndSanitizer` exposes only `approve(address,uint256)`.

That is a stronger barrier than the “not configured yet” precondition that gates every other finding in this review, and the distinction is the reason this is rated well below #8 despite sharing a root cause: #8 needs configuration steps that *are* the act of enabling an already-provisioned feature, whereas this needs **code that nobody has written**. Nothing obliges anyone to write it, either — PAXG’s `approve` accepts a non-zero-to-non-zero overwrite (verified on a mainnet fork: `approve(spender, 100e18)` then `approve(spender, 200e18)` succeeds and leaves `200e18`), so `approve` alone covers every swap flow and no swap route requires an `increaseApproval` leaf.

It is filed at Low rather than Informational on three grounds:

1. **The defect is present tense.** A shipped security control enumerates a function that does not exist on two of its three subjects. That is true at this commit, independent of any future leaf, and it is a factual error in the guard rather than a hypothesis about an attacker.
2. **The natural remediation path arms it.** The obvious response to discovering that PAXG has no `increaseAllowance` is to add the decoder entry — at which point the sweep silently fails to keep up.
3. **It changes another finding's premise.** #2's Informational rests on the assumption this removes.

Escalation trigger. This becomes Medium or higher the moment any production decoder exposes `increaseApproval` on a basket token, at which point the impact below applies with no further precondition. That transition would be silent: nothing in the codebase ties a decoder's function list to this guard's selector list, and no test would fail. If the selector set is not fixed, that coupling should at least be recorded where whoever extends the decoder will see it.

Impact

Low, as a function of a high ceiling gated behind code that does not exist.

Were the selector reachable, an allowance that survives the batch is spendable at any later time, outside the balance-verification window entirely: no snapshot, no delta bound, no value invariant, no subsidy. `increaseApproval`'s amount would not be committed by a minimal decoder (the analogous `approve` decoder commits only the spender), so the surviving allowance can be unbounded and the loss ceiling is the vault's full balance of that token. Both actors that could reach it are inside the stated threat model: a compromised strategist, or a venue that is upgraded or turns malicious after being Merkle-approved — the case the sweep exists to neutralise. The spender need not be attacker-controlled; it only needs to be a `(token, spender)` pair already present in the production root.

The rating is not a discount for the code being pre-launch. `manageRoot[EE]` is zero, so nothing in this audit is reachable today; that applies to every finding equally and cannot separate them. What lowers this one specifically is the missing decoder — a barrier no other finding in this review has.

Independently of the rating, the guard's second arm being dead code on two of three basket tokens is a correctness defect in shipped code, and the accompanying test asserts coverage that does not exist. Both are worth fixing on their own terms.

Recommendation

The narrow fix is to add `increaseApproval(address,uint256) (0xd73dd623)` — and `decreaseApproval(address,uint256) (0x66188463)` for symmetry — to the selector set, and to audit each basket token's actual allowance surface at configuration time rather than assuming the OpenZeppelin one. A comment next to `INCREASE_ALLOWANCE_SELECTOR` recording that PAXG and USDG do not implement it would stop that arm reading as coverage it does not provide.

The general fix is to stop matching calldata and assert allowance state instead. Snapshot `allowance(boringVault, spender)` for each (basket token × watched spender) pair before

`_manageVaultWithMerkleVerification`, re-read after, and require

`allowanceAfter <= allowanceBefore`

”Must not increase” rather than ”must be zero” preserves standing approvals that predate the batch (a master approval to Permit2, for instance) while forbidding the batch from creating or raising exposure by any route — this selector, the Permit2 ledger in #2, the nested layer in #8, and anything a future integration introduces. It needs an owner-configured `watchedSpenders` set with the same multisig-only access as `basketTokens`, and its completeness has to be bound into Merkle root rotation: any spender entering the root must simultaneously enter `watchedSpenders`.

Whichever is chosen, add an integration test against real token bytecode (a fork test, or a mock whose allowance surface is copied from PAXG) rather than a mock that implements `increaseAllowance`.

Developer Response

Fixed. The `increaseApproval` function selector is now detected and triggers a dangling approval check.

2.7.3 Deployment salts encode a zero deployer address, allowing anyone to front run the deployment

`DeployBoringVaultAndManager` declares its five `CreateX` salts as state-variable initializers, and Solidity evaluates those before the base constructor body assigns `broadcaster`. Every salt therefore encodes `address(0)` as its deployer — the permissionless form, in which `CreateX` does not fold `msg.sender` into the guarded salt. Any account can recompute the five deployment addresses from the name strings committed in the repository and occupy them first.

Technical Details

`DeployBoringVaultAndManager` deploys five contracts through `CreateX` using `deployCreate3`, which derives each contract’s address from a salt rather than from its bytecode, so the deployment addresses are known in advance.

`CreateX` applies access control to salts based on their leading bytes. Where the first twenty bytes hold an address equal to `msg.sender`, `CreateX` folds the sender into the salt it actually uses, so a different caller supplying the same raw salt arrives at a different address. Where the first twenty bytes are zero, that step is skipped and the raw salt is hashed alone, so every caller supplying it derives the same address. This is documented in the `CreateX` repository under permissioned deploy protection: <https://github.com/pcaversaccio/createx> — the deployed contract used here is

<https://etherscan.io/address/0xba5Ed099633D3B313e4D5F7bdc1305d3c28ba5Ed>

The script builds its salts through a helper that packs the supplied deployer address, a cross-chain protection flag, and eleven bytes of name-derived hash:


```

1 // script/Base.s.sol:141-155
2 function makeSalt(
3     address deployer,
4     bool isCrosschainProtected,
5     string memory nameEntropy
6 )
7     internal
8     pure
9     returns (bytes32)
10 {
11     bytes1 crosschainProtectionFlag = isCrosschainProtected ? bytes1(0x01) : bytes1(0x00
12 );
13     bytes32 nameEntropyHash = keccak256(abi.encodePacked(nameEntropy));
14     bytes11 nameEntropyHash11 = bytes11(nameEntropyHash);
15     return bytes32(abi.encodePacked(deployer, crosschainProtectionFlag,
16     nameEntropyHash11));
17 }

```

Passing `broadcaster` as `deployer` indicates the permissioned form is intended.

The salts are declared as state variables on the deploying contract, which inherits `BaseScript`:

```

1 // script/deploy/DeployBoringVaultAndManager.s.sol:17-31
2 contract DeployBoringVaultAndManager is BaseScript {
3     ...
4     bytes32 SALT_ROLES_AUTHORITY = makeSalt(broadcaster, false, "
5     PaxgyFundsAutomationVault: RolesAuthority");
6     bytes32 SALT_BORING_VAULT = makeSalt(broadcaster, false, "PaxgyFundsAutomationVault:
7     BoringVault");
8     bytes32 SALT_MANAGER_WITH_MERKLE_VERIFICATION =
9     makeSalt(broadcaster, false, "PaxgyFundsAutomationVault:
10    ManagerWithMerkleVerification");
11    bytes32 SALT_EQUIVALENT_EXCHANGE_UMANAGER =
12    makeSalt(broadcaster, false, "PaxgyFundsAutomationVault:
13    EquivalentExchangeUManager");
14    bytes32 SALT_PAXG_USD_RATE_PROVIDER =
15    makeSalt(broadcaster, false, "PaxgyFundsAutomationVault: PaxgUsdRateProvider");

```

`broadcaster` is declared and assigned in the parent contract:

```

1 // script/Base.s.sol:32
2 address internal broadcaster;

4 // script/Base.s.sol:49-60 (inside the BaseScript constructor)
5 constructor() {
6     address from = vm.envOr({ name: "ETH_FROM", defaultValue: address(0) });
7     if (from != address(0)) {
8         broadcaster = from;
9     } else {
10         mnemonic = vm.envOr({ name: "MNEMONIC", defaultValue: TEST_MNEMONIC });
11         (broadcaster,) = deriveRememberKey({ mnemonic: mnemonic, index: 0 });
12     }
13     ...
14 }

```

Solidity evaluates a derived contract's state variable initializers before the base constructor body executes. The five `makeSalt` calls therefore run first, while `broadcaster` still holds its default value of `address(0)`; the constructor assigns the real address afterwards.

earlier components deployed and later ones blocked, requiring cleanup of partially deployed state.

The attack yields no gain to the attacker and costs mainnet gas. At the time of review all five addresses remain unoccupied.

Recommendation

Assign the salts after `broadcaster` has been set, rather than as state variable initializers. Moving the five assignments to the top of `run()` is sufficient and preserves the existing name entropy.

```
1 // before – evaluated during construction, broadcaster is still address(0)
2 contract DeployBoringVaultAndManager is BaseScript {
3     bytes32 SALT_BORING_VAULT = makeSalt(broadcaster, false, "PaxgyFundsAutomationVault:
    BoringVault");

5     function run() public broadcast {
6         ...
7     }
8 }

10 // after – evaluated during run(), broadcaster holds the deployer address
11 contract DeployBoringVaultAndManager is BaseScript {
12     bytes32 SALT_BORING_VAULT;

14     function run() public broadcast {
15         SALT_BORING_VAULT = makeSalt(broadcaster, false, "PaxgyFundsAutomationVault:
    BoringVault");
16         ...
17     }
18 }
```

Alternatively, declare each salt as a constant containing the deployer address as a literal. That pattern is already used in `script/deploy/DeployPaxgXauRateProvider.s.sol` and is unaffected by this issue.

Before broadcasting, assert that the first twenty bytes of each salt equal `broadcaster`. A check of this form would surface the condition without requiring the initialization order to be reasoned about:

```
1 require(address(bytes20(SALT_BORING_VAULT)) == broadcaster, "salt not deployer-prefixed"
);
```

Note that CreateX does not reject a caller supplying a salt that encodes an address other than its own; it derives a different address for that caller. Protection comes from the sender being folded into the derived address, not from rejection.

Developer Response

Fixed for PaxgyFundsAutomationVault.

2.8 Gas Savings Findings

None.

2.9 Informational Findings

2.9.1 `execute`'s snapshot comment claims an alignment guarantee the code does not provide

`execute`'s opening comment states that snapshotting the basket *"guarantees ... `allowableTokenDelta[i]`, `tokens[i]` and the `oracleFlags` bit `i` can never drift out of alignment"*. Only `tokens` is snapshotted; `oracleFlags` and `tokenOracles` are read from storage after the external batch. Production authority configuration makes the drift unreachable, since `setBasketTokens` carries no role capability and stays owner-only. The finding is that a security-relevant comment attributes correctness to a snapshot that was never taken, while the real guarantee comes from an access-control setting a later change could relax.

Technical Details

`execute` opens by snapshotting the basket, and explains why:

```
1 // Read the basket once into memory. The set should not change mid-execution, but
  // snapshotting it
2 // guarantees every loop below indexes the same tokens, so `allowableTokenDelta[i]`, `
  // tokens[i]` and
3 // the `oracleFlags` bit `i` can never drift out of alignment.
4 address[] memory tokens = basketTokens.values();
```

Only `tokens` is snapshotted. The other two values the comment binds to it are read from storage after the external batch:

Value	read at	relative to the batch
<code>tokens (basketTokens.values())</code>	<code>:277</code>	before
<code>allowableTokenDelta[i]</code>	<code>calldata</code>	fixed
<code>oracleFlags</code>	<code>:351</code>	after
<code>tokenOracles[token]</code>	<code>:366</code>	after

Both live inside `_checkAndValueBasket`, which runs after `_manageVaultWithMerkleVerification`. If the basket configuration changed during the batch, `tokens[i]` and the flag for index `i` would come from different generations of the basket — the drift the comment says cannot happen.

The consequence would not be a revert but a silent mispricing, since `_checkAndValueBasket` selects between the 1:1 path and the oracle path purely from the flag bit:

```
1 uint256 baseRate = NORMALIZED_ONE;
2 uint8 rateDecimals = uint8(NORMALIZED_DECIMALS);
3 if (_hasOracleFlag(flags, i)) { ... baseRate = config.rateProvider.getRate(); ... }
```

An oracle-priced token whose bit is lost is valued at exactly one reference unit per whole token — \$1 instead of roughly \$4,300 for PAXG, in both `totalBefore` and `totalAfter`, distorting the invariant rather than tripping it.

Unreachable in production. Only a reentrant `setBasketTokens` during the batch could desynchronize them, and `setBasketTokens` is `requiresAuth` with no role capability. Checked against the live authority `0xC051ab4Cf4Ae8314CEad1126208CfFE96Bd38969` for the UManager at `0x702c68314509e8F683F8D63b3dED336890341624`, selector `0x52eaf12a`:

```
isCapabilityPublic(EE, 0x52eaf12a)      = false
getRolesWithCapability(EE, 0x52eaf12a)  = 0x00...00
EE.owner()                             = 0x00000000000417626Ef34D62C4DC189b021603f2F  (P
```

which matches the deploy script's stated intent at

[DeployBoringVaultAndManager.s.sol:124-132](#): *"setBasketTokens is given no role capability, so it stays owner-only (the multisig, after the transfer below)." The BoringVault cannot call it, so no batch can reenter it.*

For completeness, the rest of `setBasketTokens` was checked and is correct: the reverse-order drain avoids the swap-and-pop out-of-bounds read; flags are built in lockstep with `add`; the three-way consistency of `isPeggedToken` / `rateProvider` / `rateDecimals` is enforced; stale `tokenOracles` entries are genuinely unreachable once their flag bit is cleared; and `MAX_BASKET_TOKENS = 256` keeps `1 <= index` in range.

Impact

Informational. None under the deployed permission model. The defect is that correctness here is guaranteed by an authority configuration while the comment attributes it to a snapshot that was never taken. A later change that relaxes the authority setup, or adds any path able to touch the basket configuration mid-batch, would silently invalidate the invariant — and a reviewer reading this comment would have no reason to look.

Recommendation

Make the code provide the guarantee the comment claims, so correctness no longer depends on access control. Snapshot the flags with the tokens and pass them down:

```
1 address[] memory tokens = basketTokens.values();
2 bytes32 flags = oracleFlags;                                // snapshot alongside `
  tokens`
3 ...
4 (uint256 totalBefore, uint256 totalAfter, uint256 subsidyRate) =
5   _checkAndValueBasket(tokens, flags, beforeBalances, allowableTokenDelta, address(
    subsidyToken));
```

`tokenOracles` is keyed by token address rather than by index, so it cannot drift out of alignment with `tokens` once the flags are pinned — a stale entry is only ever read when its flag bit is set, which the snapshot fixes.

If the extra stack slot is a problem, the alternative is to weaken the comment to say alignment holds because `setBasketTokens` is owner-only and unreachable from a batch. That ties a code comment to a deployment setting, which is the weaker of the two options.

Developer Response

Fixed. The comment no longer makes this alignment guarantee, and instead describes configurations that would allow misalignment as invalid.

2.9.2 The subsidy is credited at the amount sent rather than the amount received, and the contract carries no unsupported-token disclaimer

Every other quantity in `execute` is derived from `balanceOf` and is immune to transfer-side deductions. `_coverShortfall` is the exception: it credits the invariant with the amount passed to `safeTransferFrom` rather than with the vault's actual balance increase, so a fee-on-transfer subsidy token would leave the vault holding less than the invariant is told it received. No basket token can trigger this today — the deployed PAXG implementation has no fee surface at all — so this is filed as an accounting asymmetry plus a missing token-assumptions disclaimer that the sibling `EquivalentExchange` contract does carry.

Technical Details

Every quantity in `execute` is derived from `balanceOf` and is therefore immune to transfer-side deductions — the delta checks and both value totals read `ERC20(token).balanceOf(boringVault)` at [:293](#) and [:356](#).

`_coverShortfall` is the one exception. It credits the invariant with the amount passed to `safeTransferFrom`, not with the vault's balance increase:

```
1 subsidyAmount = _referenceValueToTokenAmount(shortfall, rate);
2 subsidyToken.safeTransferFrom(subsidyPayer, boringVault, subsidyAmount);

4 // Re-value the actual amount transferred for accounting.
5 subsidyAmountNormalized = _tokenAmountToReferenceValue(subsidyAmount, rate);
```

The comment says *"the actual amount transferred"*, but `subsidyAmount` is the amount **requested**. With a fee-on-transfer subsidy token deducting `f`:

```
vault receives           : subsidyAmount - f
invariant is credited with: subsidyAmount
net over-credit          : f, valued at `rate`
```

and `execute` then does `totalAfter += subsidyAmountNormalized` at [:312](#), so the value assertion can pass on a batch where the vault really lost value. The availability guard at [:419](#) does not catch it — it compares the payer's balance and allowance, both pre-transfer quantities.

The same substitution breaks under a rebasing subsidy token. Note `boringVault == subsidyPayer` is already rejected at [:286](#), which shows the authors did reason about transfers that do not move real value; this is the remaining case of that class.

The exploit premise does not currently hold. It would be natural to rest this on PAXG having a configurable transfer fee currently set to zero — that is not true of the deployed PAXG implementation. Probing `0x45804880De22913dAFE09f4980848ECE6EcbAf78`:


```

feeRate()          -> reverts, empty returndata (selector absent)
feeParts()         -> reverts, empty returndata
feeController()    -> reverts, empty returndata
getFeeFor(uint256) -> reverts, empty returndata

```

There is no fee surface on the current implementation, and USDC and USDG take no transfer fee either. **No basket token as configured today can trigger this**, which is why it is filed as Informational rather than as a value-invariant bug.

What remains is a standing assumption that is nowhere written down. PAXG and USDG are upgradeable Paxos proxies, and `setBasketTokens` lets the owner add arbitrary tokens later.

The assumption is documented on the sibling contract but not this one.

[EquivalentExchange.sol:25-27](#) states:

```

//      Security assumptions:
//      - All listed tokens must implement standard ERC20 behavior, including `decimals()
//      - Fee-on-transfer and rebasing tokens are not supported.

```

`EquivalentExchangeUManager` has a full "Oracle security assumptions" block but no equivalent "Token assumptions".

Impact

Informational. No impact at the current configuration. If a fee-on-transfer or rebasing token were ever admitted to the basket and used as `subsidyToken`, the value invariant would over-credit the vault by the deducted amount on every subsidised batch — silently, since the assertion would still pass and the `Executed` event would report the requested subsidy figure.

Recommendation

Two independent changes, either of which closes the gap; both are cheap.

Measure the credit instead of assuming it:

```

1 uint256 balBefore = subsidyToken.balanceOf(boringVault);
2 subsidyToken.safeTransferFrom(subsidyPayer, boringVault, subsidyAmount);
3 uint256 received = subsidyToken.balanceOf(boringVault) - balBefore;
4 if (_tokenAmountToReferenceValue(received, rate) < shortfall) revert InsufficientSubsidy();
5 subsidyAmountNormalized = _tokenAmountToReferenceValue(received, rate);

```

The subsidy token's `balanceOf` has already been read once in `_checkAndValueBasket`, so the extra cost is one `staticcall`.

State the assumption, alongside the existing "Oracle security assumptions" block and matching the sibling contract's wording:

Token assumptions:

- All basket tokens must implement standard ERC20 behavior, including `decimals()`.
- Fee-on-transfer and rebasing tokens are not supported, as basket accounting and the subsidy both assume a transfer moves exactly the requested amount.

Also fix the comment at :425, which describes `subsidyAmount` as "the actual amount transferred".

Developer Response

Fixed. A disclaimer about fee-on-transfer tokens has been added. We understand that PAXG, a token which we support, is fee-on-transfer, but the fee is currently set to zero.

2.9.3 The `_unitRate` divide-branch safety argument does not hold for a mixed basket

`_unitRate` documents its flooring branch as unconditionally safe — *"only understates value and over-pulls subsidy, both safe"*. That holds for a single-token basket and for a basket where every token is understated in the same proportion, but not when only some tokens take the divide branch: a batch that sells an understated token and buys an accurately priced one makes the measured post-batch total larger than reality, so the value invariant can pass on a real loss. The deployed configuration never reaches the branch, but `setBasketTokens` is owner-configurable and this comment is what a future configurer would rely on.

Technical Details

`_unitRate` documents its flooring branch as unconditionally safe:

*"...a lossless multiply when that exponent is ≥ 0 (the common case, e.g. 8-dec rate + 6-to-18-dec token), else a divide that can floor — **which only understates value and over-pulls subsidy, both safe.**"*

```
1 uint256 shrink = uint256(rateDecimals) + tokenDecimals;
2 uint256 grow = 2 * NORMALIZED_DECIMALS;           // 36
3 if (shrink <= grow) {
4     return rate * (10 ** (grow - shrink));         // lossless
5 }
6 return rate / (10 ** (shrink - grow));             // floors
```

The conclusion holds for a single-token basket, and for a basket where every token is understated in the same proportion. It does not hold when only *some* basket tokens take the divide branch.

Let token `X` take the divide branch, so its computed unit rate is `r' < r` (with `r` exact), while token `Y` is priced exactly at `rate_y`. Write `Bx`, `By` for pre-batch balances and `Ax`, `Ay` for post-batch:

$$\begin{aligned} \text{measured change} &= (Ax - Bx) \cdot r' + (Ay - By) \cdot \text{rate_y} \\ \text{true change} &= (Ax - Bx) \cdot r + (Ay - By) \cdot \text{rate_y} \end{aligned}$$

For a batch that sells `X` and buys `Y`, $(Ax - Bx) < 0$, and since $r' < r$:

$$(Ax - Bx) \cdot r' > (Ax - Bx) \cdot r \quad \square \quad \text{measured change} > \text{true change}$$

The understatement discounts the outflow but nothing else, so the invariant sees a better trade than actually occurred and can pass on a real loss. "Understating value" is conservative only when applied uniformly across both sides of the comparison; here it is applied to one leg.

The existing guard does not cover this. `if (rate == 0) revert InvalidRate(token)` catches only a rate flooring all the way to zero, not a partial floor.

Not reachable in the current configuration. For the live basket:

Token	rateDecimals	decimals()	shrink	branch
USDC	18 (pegged)	6	24	multiply
USDG	18 (pegged)	6	24	multiply
PAXG	8	18	26	multiply

All below `grow = 36`, so the divide branch is unreachable. It would take a token/oracle pair whose decimals sum above 36 — an 18-decimal rate provider on a 24-decimal token, or a 27-decimal (Ray-style) rate on an 18-decimal token.

Impact

Informational. None at the deployed configuration. The exposure is that the comment states an unconditional safety property, so a future basket change introducing a high-decimal token plus a high-precision oracle would be reviewed against a guarantee that does not hold — and `setBasketTokens` is owner-configurable. The resulting mispricing would be silent rather than a revert.

Recommendation

Either qualify the comment —

```
// ... else a divide that can floor. NOTE: flooring is only conservative when every basket
// understated in the same proportion. In a mixed basket, selling a floored token for an
// one inflates totalAfter relative to reality and can let the value invariant pass on a
```

— or, preferably, remove the ambiguity by rejecting the case outright, since it is unreachable today anyway:

```
1 if (shrink > grow) revert UnsupportedRateScale(rateDecimals, tokenDecimals);
```

If the branch is kept, add a `setBasketTokens` check that either all tokens or no tokens fall into it, plus a test covering a mixed basket where the floored leg is the one being sold.

Developer Response

Fixed by blocking `tokenDecimals + rateDecimals > 36`.

2.9.4 PaxgXauRateProvider bounds each feed's age but never the skew between them

`PaxgXauRateProvider.getRate()` divides two independent Chainlink feeds, applies one absolute staleness check to each leg, and never constrains the gap between their timestamps. Measured over 81.7 days of mainnet history (400 PAXG/USD rounds, 2,600 XAU/USD rounds, cross-validated against two independent RPC endpoints), that gap has a median of 121 minutes, exceeds 12 hours in 19.9% of update events, and reaches 24.00 hours — while the per-leg check passed at **100%** of those events and, on the observed history, never fires at all. Both fee modules gate fee-vs-no-fee on the sign of `getRate() - PEG_PRICE`, a quantity whose median magnitude is 16.6 bps — smaller than a single update step of either leg (51.6 bps for PAXG/USD, 31.8 bps for XAU/USD).

Technical Details

`PaxgXauRateProvider.getRate()` builds a cross-rate by dividing two independent feeds. It applies one absolute staleness check to each leg and nothing else:

```
1 (, int256 _xauUsd,, uint256 lastUpdatedAtXau,) = XAU_USD_FEED.latestRoundData();
2 if (block.timestamp - lastUpdatedAtXau > MAX_TIME_FROM_LAST_UPDATE) revert
  MaxTimeFromLastUpdatePassed(...);
3 if (_xauUsd <= 0) revert InvalidPrice(_xauUsd);

5 (, int256 _paxgUsd,, uint256 lastUpdatedAtPaxg,) = PAXG_USD_FEED.latestRoundData();
6 if (block.timestamp - lastUpdatedAtPaxg > MAX_TIME_FROM_LAST_UPDATE) revert
  MaxTimeFromLastUpdatePassed(...);
7 if (_paxgUsd <= 0) revert InvalidPrice(_paxgUsd);

9 rate = (_paxgUsd.toUint256() * (10 ** RATE_DECIMALS)) / _xauUsd.toUint256();
```

`abs(lastUpdatedAtPaxg - lastUpdatedAtXau)` is never constrained. The threshold is shared but its derivation is argued for one feed only:

```
1 // The Chainlink PAXG/USD feed's heartbeat is 86400s (24h); we add 100s to account for
  block delay.
2 uint256 constant MAX_TIME_FROM_LAST_UPDATE = 86_500;
```

XAU/USD (`0x214eD9Da11D2fbe465a6fc601a91E62EbEc1a0D6`) is a separate aggregator with its own node set, deviation threshold and heartbeat. Nothing in the contract, the script or the tests establishes that the two update together. Note also that the already-deployed sibling provider `0x169c04078EAB33D8964EFE7DFEDD96fc761DC046` reports `MAX_TIME_FROM_LAST_UPDATE() = 90000`, wider still than the 86,500 the scripts use.

Measurement

Round history was walked back on both proxies via `getRoundData`, 400 rounds for PAXG/USD (82.5 days) and 2,600 for XAU/USD (134.9 days), giving an 81.7-day common window containing 2,098 update events. A 30-round sample of each series was re-fetched from a second, independent RPC endpoint; all 60 agreed.

Skew observed at each update event, `abs(lastUpdatedAtPaxg - lastUpdatedAtXau)`:

statistic	value
median	2.02 h
p75	9.24 h
p90	17.93 h
p99	23.49 h
max	24.00 h
> 6 h	31.5% of events
> 12 h	19.9% of events
> 18 h	9.9% of events
both legs pass the 86,500 s check	100% of events

The last row is the finding in one line. The two legs are routinely half a day apart and the contract's only temporal control never notices.

The per-leg check never fires, and has almost no margin

Across 2,998 observed inter-update intervals, neither feed ever exceeded 86,500 s:

feed	intervals	max observed gap
PAXG/USD	399	86,448 s (24.013 h)
XAU/USD	2,599	86,496 s (24.027 h)

So the check is simultaneously too loose for its stated purpose — it permits ~24 h of skew — and on a hair trigger for liveness: XAU/USD came within **4 seconds** of tripping it. A trip reverts `getRate()`, and because `WithdrawQueue.processOrders` calls the withdrawal module in a loop *outside* its `try/catch` (the module's own comment at [lines 132-136](#) records this), one late round reverts the entire withdrawal batch.

The decision variable is smaller than one feed step

Both modules branch on which side of `PEG_PRICE = 1e18` the rate falls on. Over the same window:

quantity	median	note
<code>abs(getRate() - PEG)</code>	16.6 bps	the decision variable
PAXG/USD step between consecutive rounds	51.6 bps	~0.5% deviation threshold
XAU/USD step between consecutive rounds	31.8 bps	

`getRate()` sits within 30 bps of the peg **62.8%** of the time and within 61 bps **94.4%** of the time. It lands above the peg 29.7% of the time. PAXG and gold are the same underlying, so the ratio is *supposed* to hug 1.0 — but that means the sign the fee modules read is finer-grained than the quantisation of either input. Which side of the peg the quotient lands on is decided by which rounds happen to be latest, not by any real PAXG:XAU dislocation.

Worked example, from the sampled history

At 2026-07-14 12:31:35Z, skew 10.27 h, both legs passing the staleness check:

	updatedAt	price	age
PAXG/USD	2026-07-14 02:15:23Z	\$4,009.51	10.27 h
XAU/USD	2026-07-14 12:31:35Z	\$4,091.77	0.00 h

<code>getRate()</code>	= 0.979897e18	(below peg)	
same PAXG round, XAU @ <code>t_paxg</code>	= 1.000549e18	(above peg)	[XAU was \$4,007.32 at <code>t_paxg</code>]
spurious depeg signal	= -206.5 bps		

Deposit side: the correction is `amount.mulDivUp(PEG - min(PEG, marketPrice), PEG)`. At the reported rate a depositor is charged **201 bps**; at the time-consistent rate they owe nothing.

Withdrawal side: the same inversion runs the other way, and that path has no backstop, as the module records at [lines 148-151](#): *"There is no user-set max-fee or per-order slippage backstop on withdrawals."* On deposits, `minimumMint` does not help in the direction that zeroes the fee, since the depositor receives *more* shares and `minimumMint` reverts only on fewer.

Direction matters and the earlier draft of this issue had it backwards. PAXG/USD is the slow leg (median gap 140 min, 23 full-heartbeat gaps in 82.5 days); XAU/USD is the fast one (median gap 22 min). The realistic case is a stale PAXG numerator over a fresh XAU denominator, which injects the gold move over the skew window into a ratio that should be invariant.

Scoping the claim honestly

Bucketing the stitching error by skew shows it does **not** grow monotonically with skew over this sample:

skew bucket	n	median bps	p90 bps
< 15 min	464	0.0	38.7
15 m – 1 h	329	29.5	62.5
1 – 3 h	285	30.2	62.2
3 – 6 h	178	30.3	61.7
6 – 12 h	149	31.2	65.8
12 – 18 h	35	16.9	64.8
> 18 h	20	1.2	33.9

The error saturates at roughly one XAU step because XAU/USD oscillates rather than trends over these horizons. Two consequences, both of which cut against a naive reading:

1. A skew bound helps but does not solve it. Going from ">15 min" to "<15 min" moves the median error from ~30 bps to 0, which is worth having, but the tail (p90 ~39 bps) survives because a single feed step can be large.
2. Conversely, **you do not need a large skew to get the error** — any skew at all admits a full step. The problem is not only that skew is unbounded; it is that the composed rate's resolution is coarser than the boundary it feeds.

Separate data-quality observation

Between `2026-08-01 03:14:47Z` and `2026-08-02 20:57:47Z` the XAU/USD feed alternated between two values ~1.25% apart across 625 rounds. This was verified against both RPC endpoints and is genuine on-chain data. It is excluded from every price statistic above. Its cause is unknown and it is not part of this finding, but any consumer of that feed should be aware of it.

Impact

The fee modules charge the wrong fee — including crossing the fee/no-fee boundary in both directions — from a rate stitched out of two different moments, in a regime where the everyday distance from the peg is smaller than the inputs' own resolution.

- **Deposits:** the depeg correction is zeroed or inflated. Zeroed, PAXG is credited above its true gold value, shares are over-minted, and existing shareholders absorb it; inflated, an honest depositor pays a fee the market does not justify (201 bps in the worked example).
- **Withdrawals:** the mirror image, with no max-fee or slippage parameter available to the user, and the withheld shares going to `feeRecipient`, which `deployment-config/paxgy.json` sets to `protocolAdmin`.

Typical per-event magnitude is tens of basis points, with a measured tail to 206 bps. This is not a tail-risk finding: it is the ordinary operating regime, with no attacker, no depeg and no market stress.

What separates it from the pricing risk already accepted (a real market price moving against a user, which reverts to peg) is that no price-plausibility control can see it. The project's own definition of reasonable is the post-deploy band at [lines 73-77](#):

```
1 require(rate >= 0.9e18 && rate <= 1.1e18, "DeployPaxgXauRateProvider: getRate() outside sane band");
```

Every reading discussed here is deep inside that band. The withdrawal module's stated mitigation — *"The backend can mitigate this by processing only when market prices are reasonably close to peg"* — fails for the same reason: these readings *are* close to peg. That is precisely the regime in which the sign is unreliable.

Reachability caveat, unchanged: neither fee module appears in `script/` or `deployment-config/` at this commit, so production wiring cannot be confirmed from the repository. They are this PR's deliverables, so deployment is the evident intent, and the rating assumes they are enabled. Unlike the configuration-gated findings in this review, nothing further has to be switched on for this one to be live — it is wrong from the first processed order.

Recommendation

Three changes, in order of value:

1. Bound the skew. Keep the per-leg staleness and positivity checks as an independent line of defence and add:

```
1 uint256 skew = lastUpdatedAtPaxg > lastUpdatedAtXau
2   ? lastUpdatedAtPaxg - lastUpdatedAtXau
3   : lastUpdatedAtXau - lastUpdatedAtPaxg;
4 if (skew > MAX_CROSS_FEED_SKEW) revert CrossFeedSkewTooLarge(lastUpdatedAtPaxg, lastUpdatedAtXau);
```

Derive `MAX_CROSS_FEED_SKEW` from the two feeds' measured joint behaviour, and record the derivation in a comment the way the `86_500` derivation is recorded. Note the tension the data exposes: PAXG/USD's median gap is 140 minutes, so a bound much below that will revert often. A skew bound and batch liveness trade directly against each other here, and that trade-off should be made explicitly rather than by omission.

2. Add a dead-band to the fee decision. This is the part a skew bound alone does not fix. Since `abs(getRate() - PEG)` is under 30 bps 62.8% of the time while one feed step is 32-52 bps, the modules should not act on the bare sign. Require the deviation to exceed a configured threshold before any dynamic fee applies:

```
1 uint256 deviation = marketPrice > PEG_PRICE ? marketPrice - PEG_PRICE : PEG_PRICE - marketPrice;
2 if (deviation <= DEPEG_DEADBAND) return fixedFeeOnly; // inside oracle resolution: no depeg signal
```

Sized above the feeds' quantisation (the data suggests ~60 bps, the p90 step), this makes the modules respond only to dislocations the oracle can actually resolve, and it is robust to both skew and single-step noise.

3. Justify the staleness threshold per feed. One feed's heartbeat argument is currently reused for a feed whose update characteristics were never examined, and the resulting bound

has 4 seconds of observed margin. Give each leg its own threshold derived from its own heartbeat, with enough headroom that a late round does not revert an entire withdrawal batch.

Add tests covering a pair where each leg passes its own age check but the pair straddles the peg, asserting that neither a deposit nor a withdrawal crosses the fee/no-fee boundary. The existing tests exercise only individually stale legs, which the contract already rejects.

Developer Response

Acknowledged. At this time it is a product requirement to keep deposits and withdrawals active at all times. The risk of oracle mispricing is mitigated by the directional limit of 1:1 PAXG:XAU valuation. We maintain control over when the swap between these assets takes place and at what price, and we expect any de-peg to be temporary, allowing us to delay execution until market pricing is acceptable.

2.9.5 UManager's `_boringVault` constructor argument is redundant

`UManager`'s constructor takes the `BoringVault` address as a separate argument even though it can be read from the manager. A deployment that supplies the wrong address yields a `UManager` whose batches execute against the manager's vault while its balance snapshots are read from an unrelated address. That address never changes, so the value invariant passes vacuously and no subsidy is ever pulled.

Technical Details

Currently, `UManager`'s constructor accepts both the manager address and the boring vault address:

```
1 constructor(address _owner, address _manager, address _boringVault) Auth(_owner,
  Authority(address(0))) {
2     manager = ManagerWithMerkleVerification(_manager);
3     boringVault = _boringVault;
4 }
```

The vault can be read from the manager.

Having it as a separate parameter means a deployment can supply the wrong address. The batch then executes against the manager's vault while balances are read from the address that was passed in. That address never changes, so the value invariant passes and no subsidy is pulled.

Impact

Informational. The current deploy script passes the correct address, so nothing is wrong at this commit. The exposure is that a redundant constructor parameter creates a silent, high-consequence misconfiguration: nothing in the constructor, in the script's post-deploy `requires`, or on the runtime path would surface a mismatch, and the failure mode is not a revert but a value invariant that holds trivially while the real vault is unprotected.

`_enforceNoDanglingApprovals` reads `allowance(boringVault, spender)` from the same field, so the approval sweep would be vacuous too.

Recommendation

```
1 -     constructor(address _owner, address _manager, address _boringVault) Auth(
   _owner, Authority(address(0))) {
2 +     constructor(address _owner, address _manager) Auth(_owner, Authority(
   address(0))) {
3         manager = ManagerWithMerkleVerification(_manager);
4 -         boringVault = _boringVault;
5 +         boringVault = address(manager.vault());
6     }
```

Developer Response

Fixed. The boringVault is now read from the manager instance to ensure correct configuration.

2.9.6 Design documentation reconciliation

Several statements in the supplied design and audit documents do not match the code at the reviewed commit: a `maxSubsidy` parameter that KDD 6 decided on but that was never implemented, a `RateLimitUManager` that was created and deleted before this commit, a KDD with no decision recorded, three deploy-script names that do not exist in the repository, fee-module contract names that differ from the code, a security note that has since been addressed, and sign-off sections still holding their template placeholders.

Technical Details

A few statements in the supplied design and audit documents don't match the code at the reviewed commit.

`maxSubsidy` was decided on but isn't in the code.

- *Design Doc — Equivalent Exchange*, KDD 6 "Per-Transaction Subsidy Limits" weighs two options and concludes: **Decision: `maxSubsidy` parameter.**
- `execute` takes no such parameter, and the identifier doesn't appear anywhere in `src/`.

`RateLimitUManager` no longer exists.

- *Audit Doc — Equivalent Exchange UManager*, section 2: "KDD 5 - rate limiting removed from the base and split into *RateLimitUManager* (EE does not use it)."
- Commit `8ef5149` created it and `4ae66fe` ("fix: remove broken *RateLimitUManager*") deleted it, both before the reviewed commit.
- KDD 5 reads "do not implement rate limits; separate rate limit functionality from existing base UManager implementation," so the absence of rate limits matches the decision. The audit document's description of where that code lives does not.
- Separately, `enforceRateLimit` came off four functions across `DexSwapperUManager` and `DexAggregatorUManager`, and their tests lost the corresponding coverage.

PAXGy Fee Module KDD 7 has no decision recorded.

KDD 7: Admin Fee Override

If collected withdrawal fees are held as vault shares, then their redemption will incur more withdrawal fees and permanently leave a non-zero dust amount in the vault.

Should the fee module include a separate feeless path for a privileged receiver?

Decision: ...

- Every other KDD in that document records a decision.
- No fee-exempt path exists in `PaxgyDynamicWithdrawalFeeModule`.

The deploy-script names in the scope table don't match the repo.

Audit Doc — *PAXGy Fee Modules*, section 3 lists three scripts:

Listed	In the repo
<code>DeployPaxgXauOracle.s.sol</code>	shipped as <code>DeployPaxgXauRateProvider.s.sol</code>
<code>DeployPaxgDepositFeeModule.s.sol</code>	not present
<code>DeployPaxgWithdrawFeeModule.s.sol</code>	not present

- The same document states "All three deploy scripts revert on any other chain," and its security notes rely on them for address validation and the feed `description()` checks.

Fee module contract names differ between the docs and the code.

- Documented as `PaxgDynamicDepositFeeModule` and `PaxgDynamicWithdrawFeeModule`.
- Implemented as `PaxgyDynamicDepositFeeModule.sol` and `PaxgyDynamicWithdrawalFeeModule.sol` — differing in the vault prefix and in `Withdraw` versus `Withdrawal`.

One security note has since been addressed.

- *Audit Doc* — *Equivalent Exchange UManager*, Security Notes: "subsidyPayer is unvalidated, so the vault can subsidize itself and fake the value invariant."
- It is now gated at [line 286](#):

```
1 if (subsidyPayer == boringVault) revert InvalidSubsidyPayer();
```

Section 8 still holds the template placeholders.

- Both design documents state that Section 8 reviewers must approve before development begins.
- In both, that section still shows the template rows `Person A` and `Person B` with neither mark selected.

Impact

Informational. No code defect. The exposure is that these documents are what a reviewer, a future maintainer, or a subsequent auditor reasons from, and in their present state several statements would lead that reader to expect controls, contracts, and deploy-time checks the reviewed commit does not contain — a per-transaction subsidy cap, a rate-limiting module, and three deploy scripts whose chain guards and feed `description()` checks the security notes rely on.

Recommendation

Reconcile the documents against the reviewed commit: record KDD 6's outcome as implemented rather than merely decided, correct the `RateLimitManager` reference to describe where rate limiting actually went, fill in the KDD 7 decision, align the deploy-script and fee-module names with the repository, drop the security note that has since been addressed, and complete the reviewer sign-off sections. Where a document deliberately describes an intended future state rather than this commit, say so in the document.

Developer Response

Fixed. Our design was correctly documented, but some discrepancies appeared in our exported PDF due to a formatting issue.

2.9.7 Permit2 allowances can persist

Pulling a token through Permit2 requires two grants: an ordinary ERC20 approval from the vault to Permit2, and an entry in Permit2's own ledger. `_enforceNoDanglingApprovals` sees the first and not the second, because it skips any call whose target is not a basket token and Permit2 is not one. The ledger entry survives the batch. It is inert while the ERC20 leg is swept to zero, but the merkle leaf commits neither the grant's amount nor its expiration, and rotating the root does not revoke grants Permit2 has already stored.

Technical Details

Pulling a token through Permit2 requires two separate grants. The first is an ordinary ERC20 approval from the vault to the Permit2 contract. The second is an entry in Permit2's own ledger, written by `Permit2.approve(token, spender, uint160 amount, uint48 expiration)`, naming the address permitted to pull.

`_enforceNoDanglingApprovals` sees the first and not the second. It skips any call whose target is not a basket token:

```
1 if (!basketTokens.contains(target)) continue;
```

Permit2 is not a basket token, so a call to `Permit2.approve` is skipped before its selector is examined. The Permit2 ledger entry it creates persists after the batch returns.

That entry is inert today. The ERC20 leg targets a basket token with the `approve` selector, so the sweep does check it and reverts the batch if a non-zero allowance to Permit2 remains at the end. With the ERC20 leg forced to zero, a surviving Permit2 entry cannot move anything.

The safety of the Permit2 leg therefore rests entirely on the ERC20 leg being swept. Two properties are worth noting alongside that:

- `UniswapUniversalStablecoinDecoderAndSanitizer.approve` returns `abi.encodePacked(token, spender)`. The merkle leaf commits which token and which spender, but not the amount or the expiration. A strategist may pass `type(uint160).max` and `type(uint48).max`.
- Nothing in the batch reads the Permit2 ledger, so the set of grants the vault has accumulated is not observable from the manager or from the emitted events.

Impact

Informational. No exploit path exists in the deployment as configured. The ERC20 approval to Permit2 cannot survive an `EquivalentExchangeUManager` batch, and without it a Permit2 grant of any size cannot transfer a single wei.

The exposure is confined to a route that establishes the ERC20 approval to Permit2 outside an EE batch. Any Permit2 grant accumulated earlier becomes live at that moment, for the full amount and until the expiration originally chosen, with no record of what has accumulated.

Worth noting alongside that: rotating the merkle root is the mechanism for withdrawing a route's authorization, and it has no effect on Permit2 state. Removing the leaf that authorized `Permit2.approve` stops new grants from being created; the grants already written to Permit2's ledger remain exactly as they were, for as long as their expiration allows. Revocation of the route and revocation of the authorization it produced are separate actions, and only the first has an obvious trigger.

Recommendation

Pass `expiration = 0` on every `Permit2.approve` the vault makes.

Permit2 treats zero as a special case and stores the current block timestamp in its place:

```
1 // If the inputted expiration is 0, the allowance only lasts the duration of the block.
2 allowed.expiration = expiration == 0 ? uint48(block.timestamp) : expiration;
```

[Uniswap/permit2, src/libraries/Allowance.sol:39-40](#)

The grant is usable for the remainder of the block that created it and expires on its own thereafter. An EE batch is atomic, so the swap the approval exists for always occurs within that block, and the change costs the intended route nothing. It also removes the need to trust the ERC20 sweep to contain the Permit2 leg, since a self-expiring grant is harmless whether or not the underlying approval is later re-established.

If the check is enforced on-chain rather than left to the root, enforce it on the expiration argument and not on the ledger. Permit2 does not clear the stored amount when a grant expires: a grant made with `expiration = 0` still reads its full amount a year later, expired but non-zero. A check that requires the ledger amount to be zero would reject the correct pattern.

Developer Response

Acknowledged. We do not expect to use Permit2 approvals alongside this system, and any newly granted ERC20 approvals to the Permit2 contract would be detected.



Paxos Labs PR 324

Completed 2026-08-15